SLIDING WINDOW MAXIMUM

Using Pointers, Dynamic Arrays and Nested Loops

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Problem Statement

Write a C program that reads the size and the elements of a variable-length integer array arr[] and an integer k such that 0 < k ≤ size. The array must be passed to a function named window_maximum() using its address, and pointers must be used to pass the array between the calling function and window_maximum().

The function window_maximum() must treat every contiguous subarray of size k as a sliding window, find the maximum element of each window from left to right, and display these maximums in order, separated by spaces. If k is invalid, it must display -1. A nested-loop approach must be used.

2. Objective

- To read a variable-length array using dynamic memory allocation (malloc).
- To pass an array to a function using pointers (call by address).
- To apply a nested-loop technique to process sliding windows.
- To validate input and handle an invalid window size.

3. Examples

The table below shows the expected behaviour of the program for the given examples.

No.	Array	k	Output
1	1 3 -1 -3 5 3 6 7	3	3 3 5 5 6 7
2	1 2 3 4 5	2	2 3 4 5
3	1 2 3	4	-1

4. Algorithm

1. Start the program and read the size of the array. If it is not positive, display an error and stop.
2. Allocate memory for size integers using malloc() and read the elements using pointer notation.
3. Read the window size k.
4. Call window_maximum(arr, size, k), passing the base address of the array.
5. In window_maximum(), if k ≤ 0 or k > size, print -1 and return.
6. Outer loop: let i run from 0 to size − k. Each value of i is the starting index of one window.
7. Set max = *(arr + i). Inner loop: let j run from i + 1 to i + k − 1 and update max whenever *(arr + j) is greater.
8. Print max followed by a space (no trailing space after the last window).
9. After the loops, print a newline, free the allocated memory and end the program.

5. Source Code

File name: window_maximum.c

```
#include <stdio.h>
#include <stdlib.h>

/* Prints the maximum of every contiguous window of size k.
   The array is received through a pointer (its address).
   Prints -1 if k is invalid. */
void window maximum(int *arr, int size, int k)
{
    int i, j, max;

    if (k <= 0 || k > size)
    {
        printf("-1\n");
        return;
    }

    /* Outer loop: starting index of each window */
    for (i = 0; i <= size - k; i++)
    {
        max = *(arr + i);

        /* Inner loop: scan the k elements of the current window */
        for (j = i + 1; j < i + k; j++)
        {
            if (*(arr + j) > max)
                max = *(arr + j);
        }

        printf("%d", max);
        if (i < size - k)
            printf(" ");
    }
    printf("\n");
}

int main()
{
    int size, k, i;
    int *arr;

    printf("Enter the size of the array: ");
    if (scanf("%d", &size) != 1 || size <= 0)
    {
        printf("Invalid array size.\n");
        return 1;
    }

    arr = (int *)malloc(size * sizeof(int));
    if (arr == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Enter %d elements: ", size);
    for (i = 0; i < size; i++)
        scanf("%d", arr + i);

    printf("Enter the value of k: ");
    scanf("%d", &k);

    printf("Output: ");
    window maximum(arr, size, k);

    free(arr);
    return 0;
}
```

6. Explanation of the Code

Header files: stdio.h is used for input and output, and stdlib.h provides malloc() and free().

Dynamic array: since the array size is decided at run time, memory is allocated using malloc(size * sizeof(int)). The pointer arr stores the base address of this memory.

Passing by address: the call window_maximum(arr, size, k) sends the address held in arr. Inside the function, the parameter int *arr receives it, and elements are accessed with *(arr + i) without using the [ ] operator.

Nested loops: the outer loop selects the start of each window (there are size − k + 1 windows). The inner loop checks the k elements of that window and keeps the largest one. The time complexity is O((size − k + 1) × k), which is O(n × k) in the worst case, and the extra space used is O(1).

Invalid k: the condition k <= 0 || k > size is checked first. If it is true, the function prints -1 and returns immediately.

7. Dry Run (Example 1)

Array = {1, 3, -1, -3, 5, 3, 6, 7}, size = 8, k = 3. Number of windows = 8 − 3 + 1 = 6.

i	Window	Elements compared	Maximum
0	[1, 3, -1]	1, 3, -1	3
1	[3, -1, -3]	3, -1, -3	3
2	[-1, -3, 5]	-1, -3, 5	5
3	[-3, 5, 3]	-3, 5, 3	5
4	[5, 3, 6]	5, 3, 6	6
5	[3, 6, 7]	3, 6, 7	7

Final output: 3 3 5 5 6 7

8. Sample Output

The program was compiled with gcc -Wall -Wextra and tested. Text in bold below represents user input.

Test Case 1

```
Enter the size of the array: 8
Enter 8 elements: 1 3 -1 -3 5 3 6 7
Enter the value of k: 3
Output: 3 3 5 5 6 7
```

Test Case 2

```
Enter the size of the array: 5
Enter 5 elements: 1 2 3 4 5
Enter the value of k: 2
Output: 2 3 4 5
```

Test Case 3 (invalid k)

```
Enter the size of the array: 3
Enter 3 elements: 1 2 3
Enter the value of k: 4
Output: -1
```

9. Conclusion

The program successfully finds the maximum element of every sliding window of size k using a nested-loop approach. It reads a variable-length array with dynamic memory allocation, passes it to window_maximum() through pointers, and correctly prints -1 when the window size is invalid. All three given examples produce the expected output. The approach is simple and suitable for beginners, though for very large arrays a deque-based method would run faster in O(n) time.